Data Science sous R

Chapitre 7 — Projet et évaluation des acquis

ESME — 2026-2027

Table des matières

7 Projet et évaluation des acquis **2**
7.1 Déroulé d'un projet : le DSLC en pratique 2
7.2 Le rendu : un notebook reproductible 3
7.3 Grille d'auto-évaluation 3
7.4 Réussir la soutenance 4
7.5 Où trouver des jeux de données 4
7.6 Fiche de révision 5
7.7 Questions de révision 6

Chapitre 7

Projet et évaluation des acquis

La dernière séance est consacrée à la **présentation des projets** et à l'**évaluation des acquis**. Le projet compte pour 50 % de la note du module : c'est l'occasion de mettre en œuvre l'ensemble du cycle de vie de la science des données sur un vrai jeu de données (AAV7).

Ce chapitre propose une **méthode de travail** pour le projet et une **fiche de révision**. Les modalités précises (sujet, constitution des groupes, dates et format de rendu, barème) sont communiquées par l'enseignant.

7.1 Déroulé d'un projet : le DSLC en pratique

Étape	Questions à se poser	Outils du cours
1. Formulation	Quelle est la question ? Pour qui ? Quelle décision éclairera-t-elle ? Les données permettent-elles d'y répondre ?	chapitre 1
2. Données	D'où viennent-elles ? Que représente chaque ligne, chaque variable ? Y a-t-il des valeurs manquantes, aberrantes, des doublons ?	`read_csv()`, `glimpse()`, `summary()`, `count()`
3. Découpage	Faut-il un jeu de test ? Quel découpage (aléatoire, temporel, par groupe) ?	`sample()`, `set.seed()`
4. Nettoyage	Quelles transformations ? Lesquelles sont des **choix** à documenter ?	`dplyr`, `tidyr`
5. Exploration	Distributions, relations entre variables, premiers constats	analyses uni- et bivariées, `ggplot2`
6. Structure	Peut-on résumer les variables ? Existe-t-il des groupes d'individus ?	`PCA()`, `kmeans()`, `hclust()`, `HCPC()`
7. Modélisation	Quelle méthode, et pourquoi ? Quel modèle de référence simple ?	`lm()`, `glm()`, `rpart()`
8. Évaluation PCS	Le résultat tient-il sur le jeu de test ? Avec d'autres choix de nettoyage ? Une autre graine ?	RMSE, matrice de confusion, ROC/AUC
9. Communication	Quel message principal ? Quelles limites ? Quelles recommandations ?	Quarto / R Markdown, graphiques soignés, Shiny

Commencez toujours par un **modèle de référence** très simple (prédire la moyenne, ou la classe majoritaire). Un modèle complexe n'a d'intérêt que s'il fait nettement mieux que cette référence.

7.2 Le rendu : un notebook reproductible

Le rapport doit pouvoir être **régénéré entièrement** à partir des données brutes en cliquant sur *Render* (Quarto) ou *Knit* (R Markdown). Une structure possible, en Quarto :

````
---
title: "Titre qui énonce la question"
author: "Noms des membres du groupe"
date: today
format:
  html:
    toc: true
    code-fold: true
---

```{r}
#| include: false
library(tidyverse)
source("R/fonctions.R")
```

# Résumé
Trois à cinq phrases : question, données, méthode, résultat principal, limite
↪ principale.

# 1. Contexte et question
# 2. Données : origine, signification des variables, qualité
# 3. Préparation : choix de nettoyage et leur justification
# 4. Exploration
# 5. Modélisation / analyse
# 6. Évaluation : prédictibilité et stabilité
# 7. Conclusions, limites et recommandations
# Annexe : sessionInfo()
````

7.3 Grille d'auto-évaluation

Avant de rendre votre projet, vérifiez chaque point.

Question et données

- ☐ La question est précise et liée à une décision réelle.
- ☐ L'origine des données, leur mode de collecte et la signification de chaque variable sont décrits.
- ☐ Les limites des données (représentativité, valeurs manquantes) sont discutées.
- ☐ Les questions éthiques et légales ont été examinées (données personnelles ? biais possibles ?).

Code

- ☐ Le notebook s'exécute de bout en bout sur un autre ordinateur (projet RStudio, chemins relatifs, packages listés).
- ☐ Les données brutes ne sont jamais modifiées à la main.
- ☐ Le code répété est factorisé en fonctions aux noms explicites.
- ☐ Les tirages aléatoires sont reproductibles (`set.seed()`).

Analyse

- ☐ Chaque méthode est justifiée par rapport à la question et au type des variables.
- ☐ Les graphiques ont un titre, des axes nommés avec unités et une légende lisible.
- ☐ Les performances sont mesurées sur des données **non utilisées** pour l'apprentissage, et comparées à une référence simple.
- ☐ Au moins une vérification de **stabilité** est présentée (autre graine, autre traitement des valeurs manquantes, autre méthode).

Communication

- ☐ Le message principal tient en une phrase.
- ☐ Les résultats sont interprétés dans les termes du domaine (en marketing : clients, ventes, campagnes), pas seulement en chiffres.
- ☐ Les limites et l'incertitude sont énoncées honnêtement.
- ☐ Les recommandations sont concrètes et réalisables.

7.4 Réussir la soutenance

- Commencer par la **question** et le **message principal**, pas par le code.
- Une idée par diapositive ; des graphiques lisibles de loin.
- Montrer le cheminement, y compris les impasses et les choix difficiles.
- Prévoir une démonstration courte (par exemple une application Shiny) si elle apporte quelque chose.
- Préparer les questions : « Pourquoi cette méthode ? », « Ce résultat est-il stable ? », « Que feriez-vous avec plus de temps ou de données ? ».

7.5 Où trouver des jeux de données

Marketing et commerce

- *Bank Marketing* (UCI) : résultats de campagnes de télémarketing d'une banque portugaise ;
- *Online Retail II* (UCI) : deux ans de transactions d'un site de vente en ligne britannique, idéal pour une analyse RFM et une segmentation ;
- *Telco Customer Churn* (IBM, disponible sur Kaggle) : résiliation d'abonnés d'un opérateur ;
- les jeux du package `ISLR` utilisés dans ces notes (`Carseats`, `OJ`, `Caravan`) et la boutique simulée du cours (`datasets`).

Données publiques

- data.gouv.fr et INSEE : données publiques françaises ;
- Eurostat : statistiques européennes ;
- Kaggle Datasets et UCI Machine Learning Repository.

Vérifiez toujours la **licence** et la **documentation** d'un jeu de données avant de l'utiliser.

7.6 Fiche de révision

7.6.1 Le langage R (chapitre 2)

Besoin	Code
assigner	`x <- 3`
type, structure	`typeof(x)`, `class(x)`, `str(x)`
vecteur, facteur	`c(1, 2)`, `factor(c("a", "b"))`
liste, data frame	`list(a = 1, b = "x")`, `data.frame(x = 1:3)`
indexer	`x[2]`, `x[x > 0]`, `df[i, j]`, `df$col`, `liste[[1]]`
condition	`if (cond) { } else { }`, `ifelse(cond, a, b)`
boucle	`for (i in seq_along(x)) { }`, `while (cond) { }`
fonction	`f <- function(x, y = 1) { ... }`
valeurs manquantes	`is.na(x)`, `mean(x, na.rm = TRUE)`
packages	`install.packages("p")`, `library(p)`, `p::f()`

7.6.2 Manipulation (chapitre 3)

Besoin	Code
lire, écrire	`read_csv()`, `read_csv2()`, `read_excel()`, `write_csv()`, `saveRDS()`
décrire	`summary()`, `quantile()`, `table()`, `freq()`
croiser	`table(x, y)`, `lprop()`, `cprop()`, `chisq.test()`, `t.test()`, `cor()`
lignes	`filter()`, `slice_max()`, `arrange(desc())`, `distinct()`
colonnes	`select()`, `rename()`, `mutate()`, `case_when()`
résumer	`group_by() |> summarise()`, `count()`
joindre	`left_join(x, y, by = c("a" = "b"))`, `anti_join()`
remodeler	`pivot_longer()`, `pivot_wider()`
générer	`seq()`, `rep()`, `rnorm()`, `sample()`, `set.seed()`

7.6.3 Graphiques et régression (chapitre 4)

Besoin	Code
base R	`plot()`, `hist()`, `boxplot()`, `barplot()`
ggplot2	`ggplot(df) + geom_point(aes(x, y, color = g)) + labs()`
barres	`geom_bar(position = "fill")`, `geom_col()`
petits multiples	`facet_wrap(vars(g))`

Besoin	Code
régression linéaire	`lm(y ~ x1 + x2, data = df)`, `summary()`, `predict()`
régression logistique	`glm(y ~ ., data = df, family = binomial)`, `exp(coef())`

7.6.4 ACP et arbres (chapitre 5)

Besoin	Code
ACP	`PCA(df, quanti.sup = , quali.sup = , graph = FALSE)`
graphiques ACP	`fviz_eig()`, `fviz_pca_var()`, `fviz_pca_ind()`
décrire les axes	`dimdesc()`
arbre	`rpart(y ~ ., data, method = "class")`, `printcp()`, `prune()`
évaluer	`table(observe, predit)`, `roc()`, `auc()`

7.6.5 Classification (chapitre 6)

Besoin	Code
standardiser	`scale(df)`
k-moyennes	`kmeans(x, centers = k, nstart = 25)`
CAH	`hclust(dist(x), method = "ward.D2")`, `cutree(cah, k)`
choisir k	`fviz_nbclust(x, kmeans, method = "silhouette")`
ACP + CAH	`HCPC(res_acp, nb.clust = -1)`
Shiny	`ui <- fluidPage(...)`, `server <- function(input, output) {...}`

7.7 Questions de révision

1. Pourquoi un résultat issu de données est-il une preuve plutôt qu'une démonstration ?
2. Quelle est la différence entre les jeux de validation et de test ? Pourquoi découper avant d'explorer ?
3. Quel découpage choisir pour prédire les ventes du mois prochain ? Pour prédire le comportement des clients d'un nouveau magasin ?
4. Donnez un exemple de fuite de données dans un projet de prédiction de l'attrition.
5. Quelle différence entre `&` et `&&` ? Entre `[ ]` et `[[ ]]` sur une liste ?
6. Pourquoi regarder à la fois la moyenne et la médiane d'un montant d'achat ?
7. Dans un test A/B, que conclure d'une différence non significative ?
8. Que fait `left_join()` si la clé n'est pas unique dans la seconde table ? Quelle jointure permet de trouver les clients sans commande ?
9. Qu'est-ce qu'un jeu de données *tidy* ?

10. Quand une variable va-t-elle dans `aes()`, et quand hors de `aes()` ?
11. Comment interpréter un coefficient de régression multiple ? Pourquoi peut-il changer quand on ajoute une variable ? Comment interpréter un rapport de cotes ?
12. Pourquoi réduire les variables avant une ACP ou un calcul de distances ?
13. Comment lire l'angle entre deux flèches du cercle des corrélations ? À quoi sert une variable illustrative ?
14. Pourquoi élaguer un arbre de décision ?
15. Pourquoi le taux de bonne classification est-il trompeur sur des classes déséquilibrées ? Que représente l'AUC ?
16. Quelles différences entre k-moyennes et CAH ? Comment choisir le nombre de classes d'une segmentation ?
17. Citez trois risques éthiques d'un projet de ciblage marketing et une précaution pour chacun.
18. Quel est le rôle de `reactive()` dans une application Shiny ?